

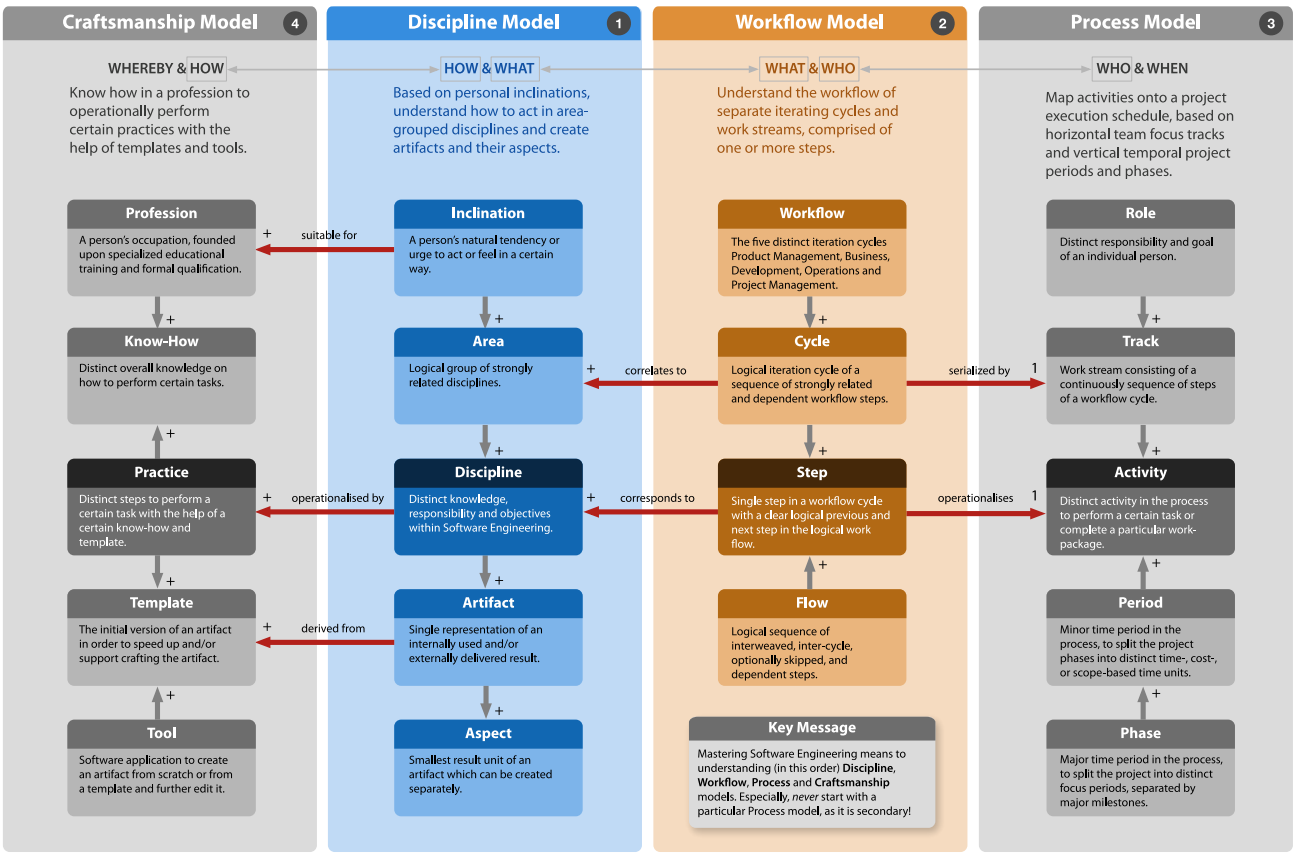
Software Engineering in Industrial Practice (SEIP)

Dr. Ralf S. Engelschall

ENGINEERING FUNDAMENTALS

Software Engineering Metamodel

TECHNISCHE UNIVERSITÄT MÜNCHEN



Intellectual content: Version 1.0.4 (2021-04-08), Authored 2021 by Dr. Ralf S. Engelschall, Graphical illustration: Version 1.0.2 (2021-04-13), Copyright © 2021 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved. Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contents only.

EF 02.1

big picture & coherency

Software Engineering kann durch ein Meta-Modell verstanden werden, das auf vier verschiedenen, aber miteinander verknüpften Modellen basiert.

Das Handwerks-Modell (**Craftsmanship Model**) ist die Basis und zielt auf das WOMIT & WIE. Es spannt den Bogen von den Berufen (**Profession**) der einzelnen Personen, deren entsprechenden **Know-Hows** und Praktiken (**Practice**) bis hin zu den zugrunde liegenden Vorlagen (**Templates**) und Werkzeugen (**Tools**).

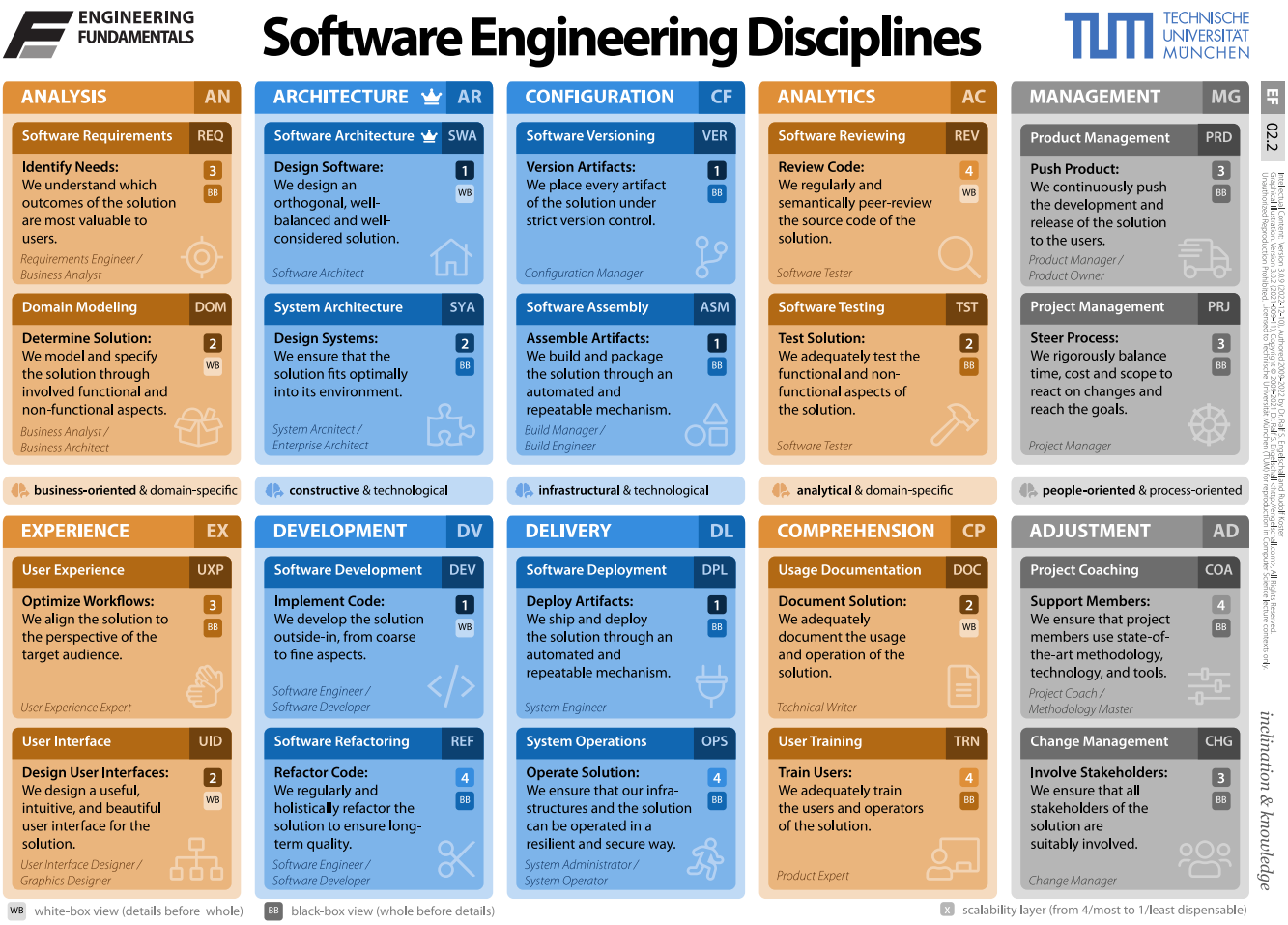
Das Disziplinen-Modell (**Discipline Model**) zielt auf das WIE & WAS. Es trennt Software Engineering in Disziplinen (**Disciplines**), die in Bereiche (**Area**) gruppiert sind und die durch die üblichen Neigungen (**Inclination**) von einzelnen Personen motiviert sind. Jede Disziplin wird dann durch Eingabe- und Ausgabe-Artefakte (**Artifact**) und deren Aspekte (**Aspect**) beschrieben.

Das Arbeitsablauf-Modell (**Workflow Model**) zielt auf das WAS & WER. Es beschreibt einen Arbeitsablauf (**Workflow**) aus Zyklen (**Cycle**), die Schritte (**Step**) enthalten. Ein Fluss (**Flow**) ist der Durchlauf durch diese Schritte über die Zeit.

Das Prozess-Modell (**Process Model**) schließlich zielt auf das WER & WANN. Es bildet Aktivitäten (**Activity**) auf einen Projektablaufplan ab, basierend auf horizontalen Bahnen (**Track**) von Rollen (**Roles**) und vertikalen Perioden (**Period**) von Phasen (**Phase**).

Fragen

- Wieviele Zyklen kennt man im Workflow Model des Software Engineering, in denen jeweils die Personen mit ähnlichen Neigungen agieren?



Software Engineering lässt sich durch 20 verschiedene Disziplinen (**Discipline**) (operationalisiert durch Input- und Output-Artefakte und deren Aspekte), die logisch in 10 verschiedene Bereiche (**Area**) gruppiert sind, und die wiederum logisch in 5 verschiedene Neigungen (**Inclination**) von einzelnen Personen gruppiert sind, verstehen.

Personen mit einer starken fachlichen und geschäftlichen (**domain-specific** and **business-oriented**) Neigung agieren in den Bereichen Analyse (**Analysis**) und Erfahrung (**Experience**) und in den entsprechenden Disziplinen Software-Anforderungen (**Software Requirements**), Fachmodellierung (**Domain Modeling**), Benutzererfahrung (**User Experience**) und Benutzungsschnittstelle (**User Interface**).

Personen mit einer starken konstruktiven und technologischen (**constructive** and **technological**) Neigung agieren in den Bereichen Architektur (**Architecture**) und Entwicklung (**Development**) und in den entsprechenden Disziplinen Software-Architektur (**Software Architecture**), System-Architektur (**System Architecture**), Software-Entwicklung (**Software Development**) und Software-Überarbeitung (**Software Refactoring**).

Personen mit einer starken infrastrukturellen und technologischen (**infrastructural** and **technological**) Neigung agieren in den Bereichen Konfiguration (**Configuration**) und Auslieferung (**Delivery**) und in den entsprechenden Disziplinen Software-Versionierung (**Software Versioning**), Software-Montage (**Software Assembly**), Software-Bereitstellung (**Software Deployment**) und Software-Betrieb (**Software Operations**).

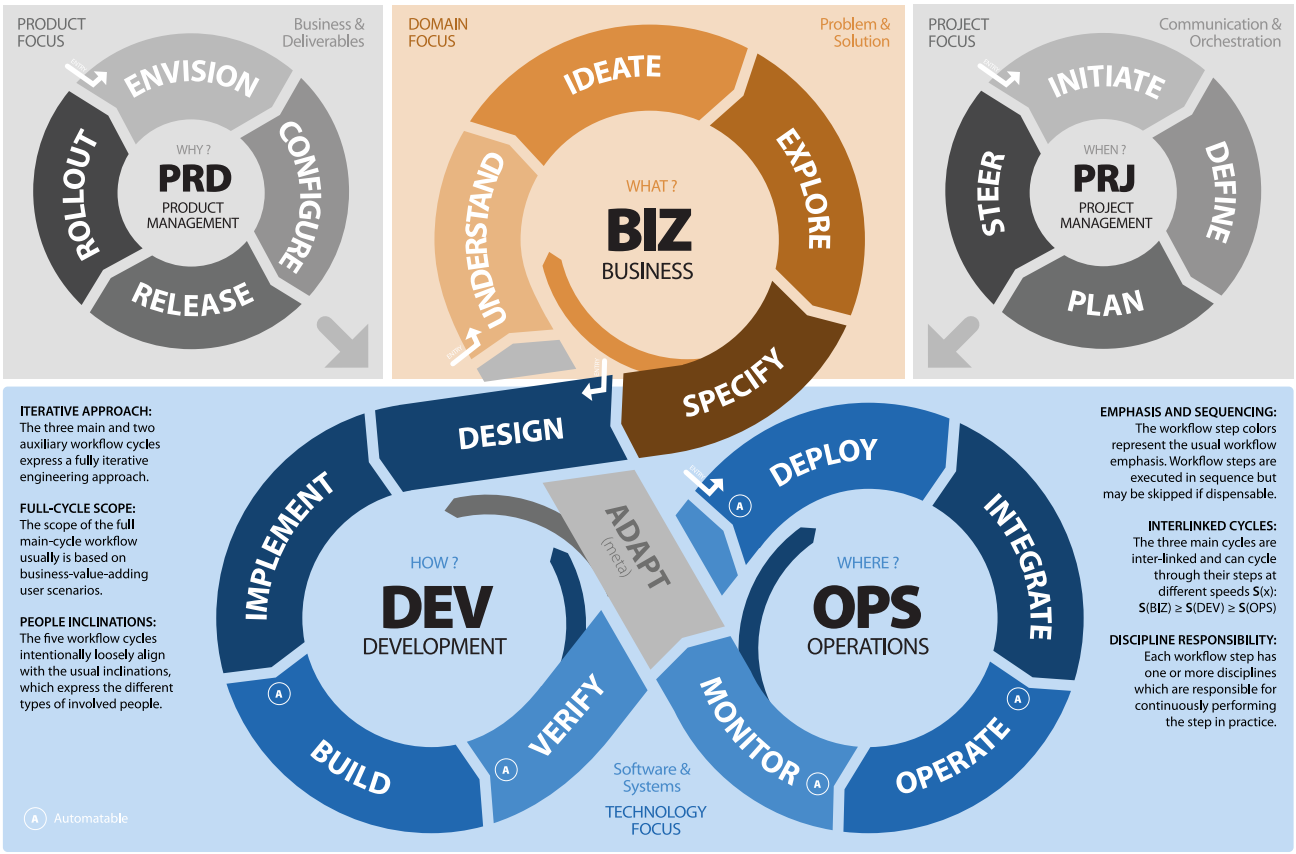
Personen mit einer starken analytischen und fachlichen (**analytical** and **domain-specific**) Neigung agieren in den Bereichen Analytik (**Analytics**) und Verständnis (**Comprehension**) und in den entsprechenden Disziplinen Software-Überprüfung (**Software Reviewing**), Software-Test (**Software Testing**), Benutzungsdokumentation (**User Documentation**) und Benutzerschulung (**User Training**).

Personen mit einer starken personellen und prozessorientierten (**people-oriented** and **process-oriented**) Neigung agieren in den Bereichen Führung (**Management**) und Anpassung (**Adjustment**) und in den entsprechenden Disziplinen Projektmanagement (**Project Management**), Projektauditierung (**Project Auditing**), Projektcoaching (**Project Coaching**) und Veränderungsmanagement (**Change Management**).

Fragen

- Welche Disziplinen im Software Engineering werden als die beiden **Königsdisziplinen** angesehen?

Software Engineering Workflow



Intellectual Content: Version 1.1 (2021-12-29), Authored 2019-2021 by Dr. Ralf T. Kneuper
Graphics: Illustration: Version 1.1.0 (2021-12-29), Copyright © 2021-2021 Dr. Ralf T. Kneuper, http://kneuper.de, All rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science for Business courses only.

Das Workflow-Modell beschreibt die Arbeitsteilung im Software Engineering. Im **Workflow** drücken drei Haupt- und zwei Neben-Workflow-Zyklen das iterative Vorgehen aus. Der vollständige Haupt-Workflow-Zyklus basiert in der Regel auf Geschäfts-wertschöpfenden Anwenderszenarien.

Die drei Hauptzyklen sind miteinander verknüpft und können ihre Schritte mit unterschiedlichen Geschwindigkeiten S(x) durchlaufen, wobei S(BIZ) größer oder gleich S(DEV) ist, welches wiederum größer oder gleich S(OPS) ist. Denn die Zyklen mit früheren Schritten sollten die Zyklen von späteren Schritten nicht ausbremsen.

Die Schrittfarben repräsentieren den Arbeitsablaufschwerpunkt. Arbeitsablaufschritte werden der Reihe nach ausgeführt, können aber übersprungen werden, wenn sie entbehrlich sind.

Die zehn Disziplinen des Software Engineering drücken die unterschiedlichen Rollen der beteiligten Personen aus. Die fünf Neigungen drücken die verschiedenen Typen der beteiligten Personen aus. Deshalb besteht der Workflow aus genau fünf Zyklen.

Fragen

- Was beschreibt das Workflow-Modell im Software Engineering?

ENGINEERING FUNDAMENTALS

Software Engineering Steps

TECHNISCHE UNIVERSITÄT MÜNCHEN



Intellectual Content: Version 1.0.1 (2021-04-13) Authored 2021 by Dr. Ralf S. Engelschall and Rudolf Koster.
Graphical Illustration: Version 1.0.1 (2021-04-13) Copyright © 2021 Dr. Ralf S. Engelschall <http://engelschall.com> – All Rights Reserved.
Unauthorized Reproduction Prohibited. Licenced to Technische Universität München (TUM) for reproduction in Computer Science lecture contents only.

Software Engineering kann auf operativer Ebene alternativ durch 20 verschiedene **Steps** verstanden werden, die innerhalb des **Software Engineering Workflow** kontinuierlich durchgeführt werden. Jeder Step gehört zu einer primär verantwortlichen Disziplin und keiner oder mehreren sekundär verantwortlichen Disziplinen.

Workflow Steps sind das geeignete Konzept, um zu verstehen, welche Aktivitäten in jeder Iteration eines **Software Engineering Prozesses** durchgeführt werden müssen.

Fragen

- Welches Konzept erlaubt einem am besten zu verstehen, welche Aktivitäten in einem Software Engineering Prozess durchgeführt werden müssen?

Software Engineering Process

1. WORKFLOW CYCLES
The workflow has five cycles which continuously iterate through their steps. Workflow steps are executed in each cycle in sequence, but may be skipped if dispensable in a particular iteration of the process. The length of an iteration is arbitrary, but can be e.g. about 1/3 of a Scrum sprint.

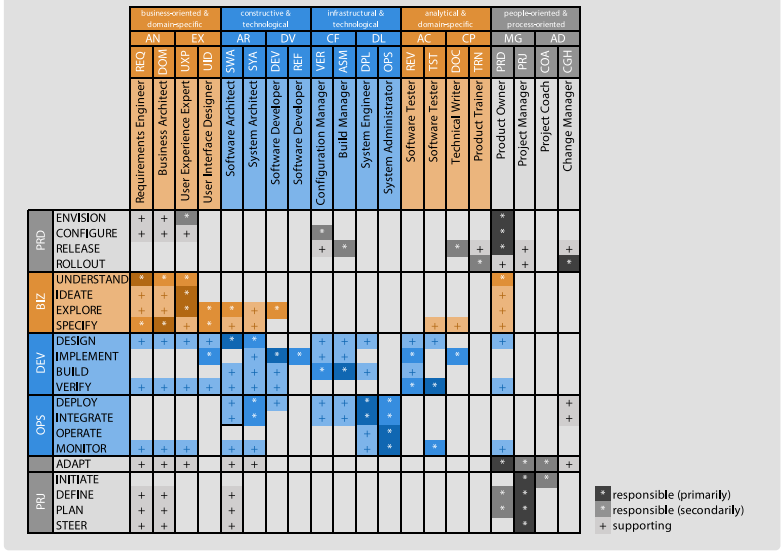
2. WORKFLOW STEPS:
The workflow steps describe a logical activity which has to be performed. Each step relates to one or more discipline areas and their corresponding disciplines, which express the operative responsibilities for each workflow step. In each discipline individual roles act.

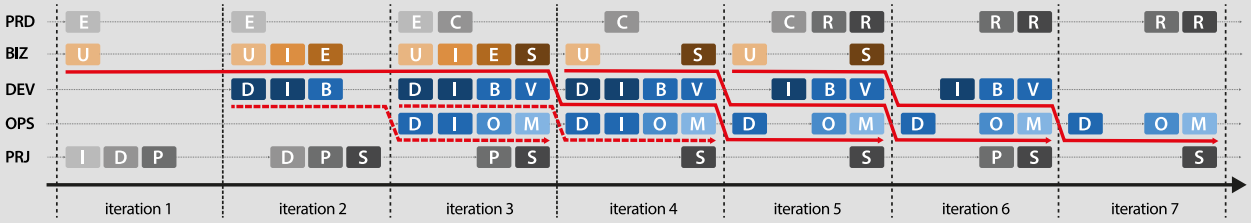
3. WORKFLOW ROLES:
The workflow roles are held by individual persons. Each role is primarily responsible for a particular workflow step. In addition, each role can be secondarily responsible for other workflow steps or at least actively support those steps.

4. PROJECT SCHEDULE:
To create a particular project execution schedule, the five cycles, their iterations and their steps have to be mapped onto a timeline. The cycles are mapped onto (horizontal) timeline tracks, the iterations are mapped onto (vertical) timeline phases, and the steps are mapped onto timeline activities.

5. PROCESS FLOWS (THE CRUX):
The activities across the cycles can (and should) be linked into individual (diagonal) waterfall-like flows, although the execution schedule, from the perspective of the cycles, is fully iterative. There are multiple such flows in parallel and they are usually highly interleaved on the project timeline in order to maximally utilize the team.

6. PROCESS ADAPTION:
In the meta-step ADAPT, the process is adapted by choosing which workflow steps are required for the next iteration. The major input for this decision is the current solution state and the feedback on it by the customer.





Der Arbeitsablauf hat fünf Hauptzyklen, die ihre Schritte kontinuierlich durchlaufen. Die Schritte des Arbeitsablaufs werden in jedem Zyklus der Reihe nach ausgeführt, können aber übersprungen werden, wenn sie entbehrlich sind.

Die Arbeitsablauf-Schritte sind mit Disziplinbereichen annotiert, um operative Verantwortlichkeiten auszudrücken. In jedem Bereich agieren mehrere Rollen.

Die Arbeitsablauf-Rollen werden von einzelnen Personen besetzt. Jede Rolle ist primär für einen bestimmten Arbeitsschritt verantwortlich. Darüber hinaus kann jede Rolle sekundär für andere Arbeitsschritte verantwortlich sein oder zumindest diese Schritte aktiv unterstützen.

Um einen bestimmten Projektablaufplan zu erstellen, müssen die fünf Zyklen, ihre Iterationen und ihre Schritte auf einer Zeitachse abgebildet werden. Die Zyklen werden auf (horizontale) Zeitleistenspuren abgebildet, die Iterationen werden auf (vertikale) Zeitleistenphasen abgebildet, und die Schritte werden auf Zeitleisten-Aktivitäten abgebildet.

Die Aktivitäten über die Zyklen hinweg können (und sollten) zu einzelnen (diagonalen) wasserfallartigen Flüssen verknüpft werden, obwohl der Ausführungsplan aus der Perspektive der Zyklen vollständig iterativ ist. Es gibt mehrere solcher Flüsse parallel und sie sind normalerweise stark auf der Projektzeitachse verschränkt, um das Team maximal auszulasten.

Fragen

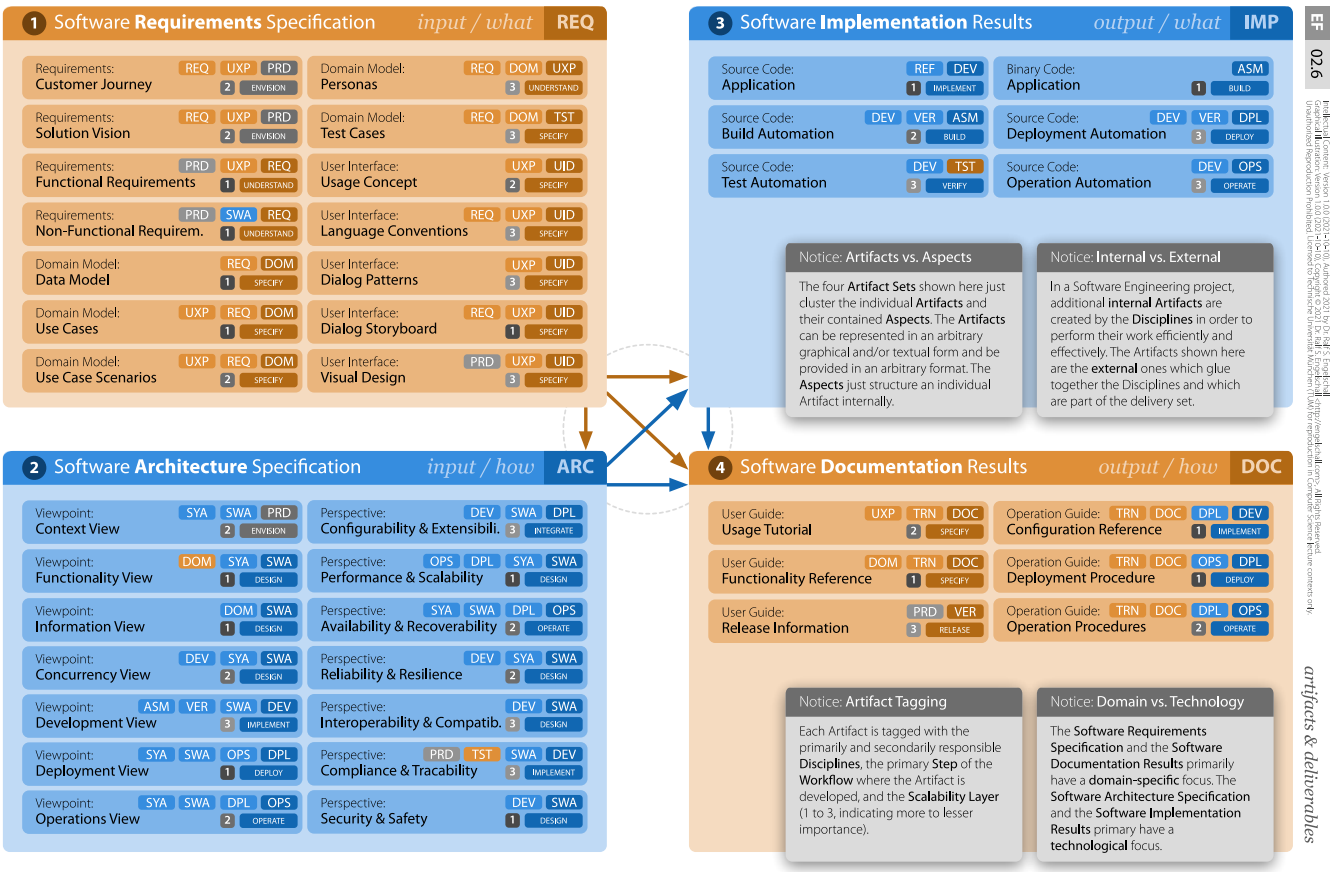
- Wie wird trotz Arbeitsteilung eine maximale Auslastung des Teams im Software Engineering erreicht?

Intellectual Content: Version 1.0.9 (2021-12-18), Authored 2019-2021 by Dr. Ralf S. Engelschall
Graphic Illustration: Version 1.1.1 (2021-12-18), Copyright © 2020-2021 Dr. Ralf S. Engelschall, http://engelschall.com, All rights reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science for Curse comments only.

ENGINEERING FUNDAMENTALS

Software Engineering Artifacts

TECHNISCHE UNIVERSITÄT MÜNCHEN



Intellectual Content: Version 1.0.0 (2021-10-10). Authored 2021 by Dr. Ralf S. Engelschall. Graphical Illustration: Version 1.0.0 (2021-10-10). Copyright © 2021 Dr. Ralf S. Engelschall <http://engelschall.com>. All Rights Reserved. Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Die vier **Artifact Sets** bündeln lediglich die einzelnen **Artifacts** und ihre enthaltenen **Aspects**. Die **Artifacts** können in beliebiger grafischer und/oder textueller Form dargestellt werden und in einem beliebigen Format bereitgestellt werden. Die **Aspects** strukturieren lediglich ein einzelnes Artifact intern.

In einem Software-Engineering-Projekt werden zusätzliche **interne** Artifacts von den **Disziplinen** erstellt, um ihre Arbeit effizient und effektiv durchzuführen. Die gezeigten Artifacts sind nur die **externen** Artifacts, welche die Disziplinen verbinden und die Teil des Lieferumfangs sind.

Jedes **Artifact** ist mit der primären und den sekundären **Disciplines**, dem primären **Step** des Workflows, in dem das Artifact entwickelt wird, und der Skalierbarkeitsstufe (1 bis 3, von mehr bis weniger wichtig) annotiert.

Die **Software Requirements Specification** und die **Software Documentation Results** haben primär einen **fachlichen** Fokus. Die **Software Architecture Specification** und die **Software Implementation Results** haben primär einen **technologischen** Fokus.

Fragen

- Welchen Fokus hat die **Software Requirements Specification**?